# HNFFS: Revisiting the NOR Flash File System

Yanqi Pan†, Zhisheng Hu†, Nan Zhang†, Hao Hu†, Wen Xia†‡, Zhongming Jiang†, Liang Shi§, and Shiyi Li†
†School of Computer Science and Technology, Harbin Institute of technology, Shenzhen, China
‡Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies
§School of Computer Science and Technology, East China Normal University, Shanghai, China

2022 IEEE 11th Non-Volatile Memory Systems and Applications Symposium (NVMSA) | 978-1-6654-5078-2/22/$31.00 ©2022 IEEE | DOI: 10.1109/NVMSA56066.2022.00012

***Abstract*—NOR flash, a type of non-volatile memory technologies, embraces its new age of IoT due to its execute-in-place (XIP) feature. Generally, there are two representative file systems designed for NOR flash: Journaling Flash File System 2 (JFFS2) and Spi Flash File System (SPIFFS). They suffer from either slow mount time, heavy foreground garbage collection (GC) overheads, poor read/write performance, or inefficient Wear-Leveling (WL), and thus they are unfriendly to IoT devices.**

**To overcome the above limitations of existing NOR flash file systems, we propose and design a Harmony NOR Flash File System (HNFFS): (1) We introduce Erasable Sector Summary (ESS) to reduce mount time by leveraging NOR flash's fast read capabilities. ESS deploys *summary node*s to avoid scanning the whole NOR flash when mounting the system. (2) We propose Adaptive Garbage Collection (AGC), which allows the NOR flash file system to adaptively trigger GC in advance and thus reduces the heavy *Foreground GC* overheads. (3) We introduce Mergeable Tree (MT) to decrease the DRAM usage for indexing while improving write performance by merging the sequential small-data write. (4) We provide Random Static Wear-Leveling (RSWL) to efficiently perform WL by combining the high performance Random WL (RWL) and the reliable Static WL (SWL).**

**Experimental results suggest that HNFFS outperforms the state-of-the-art NOR flash file systems. Moreover, the evaluation of RSWL also shows the optimal tradeoff between performance and wear evenness.**

***Index Terms*—NOR flash memory, Flash file System, Embedded systems**

## I. Background and Introduction

Flash memory has become an increasingly significant component of non-volatile storage media due to its compact size, shock resistance, and low power consumption [2]. NOR and NAND flash are two major flash memory products that are widely adopted in many applications, which have attracted many researches, such as [7], [8], [22], etc. When compared to NAND flash memory, NOR flash memory suffers a higher cost and lower storage capability, and usually used in the resource-restricted embedded devices [3]. However, NOR flash cannot be substituted by NAND due to its XIP feature required by many IoT scenarios [4].

NOR flash is normally arranged into sectors, where each sector is byte-addressable and typically 64–128KB. In contrast to NAND flash [24], [25], NOR flash is reliable and more suitable for extreme environment. However, due to it is a form of Electrically Erasable Read Only Memory (EEPROM) with limited erasing life-cycle, out-of-place updates must be supplied [5]. Note that, the erasing life-cycle of NOR flash is much shorter than that of NAND flash. In that case, reliable wear-leveling algorithm is important. TABLE I shows the general comparison between NOR flash and NAND flash.

TABLE I
General comparison between NOR flash and NAND flash

| Properties | NOR Flash | NAND Flash |
|---|---|---|
| Part Number | MT28F128J3 [17] | MT29F2G08AAD [18] |
| Capacity | Low (16MB) | High (2GB) |
| eXecute In Place | Support | No support |
| Read/Write Cycle Time | 150ns | 25us/220us |
| Erasing Life-cycle | 10,000–100,000 | 100,000–1,000,000 |
| Reliability | High (almost no bad sectors) | Low |

File systems are suitable for accessing NOR flash due to their general interface. JFFS2 and SPIFFS are two representative file systems designed for NOR flash [1], [6], [9]. The former uses unbounded memory to perform efficient I/O; while the latter limits its memory usage, which is friendly to DRAM-restricted IoT devices. However, JFFS2 suffers slow mount time, heavy foreground garbage collection (GC) overheads, explosively increasing memory usage, and poor evenness Wear-Leveling (WL); while SPIFFS is inefficient when it comes to I/O performance due to poor memory usage.

In this paper, we design and implement an efficient NOR flash file system, HNFFS, which is inspired by revisiting JFFS2 and SPIFFS and targets at tackling the aforementioned issues. Specifically, contributions of this work are five folds:

- **To reduce mount time**, HNFFS introduces Erasable Sector Summary (ESS) by making full use of read capability of NOR flash. ESS reserves *summary node* at the end of each sector which contains meta information of all the data in this sector. In that case, HNFFS is allowed to mount the system by scanning *summary node*s rather than the whole NOR flash, see Section III-B.
- **To mitigate heavy *Foreground GC* overheads**, HNFFS proposes an Adaptive Garbage Collection (AGC) mechanism, which utilizes the idle time of the file system and puts GC operation in advance. Moreover, by considering limited erasing life-cycle of NOR flash and restricted resources of IoT devices, HNFFS triggers background GC when the system's utilization is $\geq 30\%$ and allows migrating one valid *node* per pass, see Section III-C.
- **To reduce the memory usage for small-data writes**, HNFFS proposes Mergeable Tree (MT). MT automatically merges the in-DRAM nodes that represents the sequential small-data writes, reducing memory usage and indexing overheads, and thus improving the read/write performance, see Section III-D.
- **To improve wear evenness**, HNFFS proposes Random Static Wear-Leveling (RSWL) algorithm, which is benefited from the high performance of Random Wear-Leveling (RWL) and the reliability of Static Wear-Leveling (SWL), see Section III-E.
- Evaluation shows that HNFFS consistently achieves better performance over the state-of-the-art NOR flash file sys-

978-1-6654-5078-2/22/$31.00 ©2022 IEEE
DOI 10.1109/NVMSA56066.2022.00012



Authorized licensed use limited to: Peking University. Downloaded on November 19,2022 at 09:09:10 UTC from IEEE Xplore. Restrictions apply.

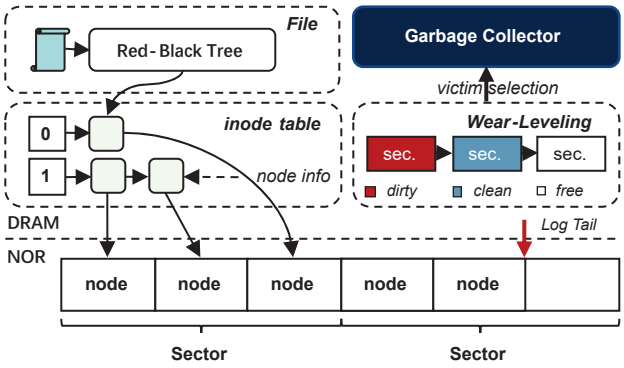


Fig. 1. HNFFS architecture

tems (JFFS2 and SPIFFS). Besides, by introducing RSWL, HNFFS also brings an optimal tradeoff between performance and wear evenness, see Section IV.

## II. Related Work and Motivation

There are several NOR flash file systems nowadays. One of them is JFFS2, which presents a series of flash file systems that tend to use unbounded memory and perform efficient I/O (such as [23], [26], [27], etc). JFFS2 is organized as a log structure [1] and provides RWL by randomly selecting victim sectors to erase. JFFS2 regards every file data as a data entity and keeps complex in-DRAM structures to index them. If there are not enough available sectors in JFFS2, GC is triggered. Generally, the design of JFFS2 suffers at least four issues, which are **either unfriendly to NOR flash or IoT devices**: (1) **Slow mount time**. JFFS2 needs to scan the whole NOR flash media to build the in-memory indexing structure. With the scaling size of NOR flash memory (like 128MB in AirPods), it takes intolerant long time to mount the system. (2) **Heavy *Foreground GC* overheads**. JFFS2 triggers GC only when the system has not enough available sectors to perform the write request, which blocks the (foreground) write path significantly and thus brings heavy overheads to the system. (3) **Exploded memory usage**. With the increment data entity to index, JFFS2 needs exploded memory to maintain its indexing structure (i.e., Red-Black tree). (4) **Poor wear evenness**. The principal of RWL is based on probability, which gains higher performance but suffers poor wear evenness. Our experiment shows that RWL is 87% less wear evenness than that of SWL.

SPIFFS is another NOR flash file system [6]. SPIFFS is designed with bounded memory in mind, so it represents the flash file systems that use restricted DRAM resource (such as [7], [16], [22], etc). The feature is widely accepted by many embedded systems, such as HUAWEI LiteOS [9]. SPIFFS provides SWL by scanning every NOR flash sector and selecting the lowest erase count sector to erase. SPIFFS is reliable but inefficient for two reasons: (1) Due to its poor bounded memory usage, SPIFFS suffers extremely low read/write performance. (2) Performing SWL consumes much of bandwidth of NOR flash, which is unfriendly for some I/O intensive IoT scenarios (e.g., 5G).

Except the NOR flash file systems aforementioned, Unsorted Block Image File System (UBIFS), which is also known as JFFS3 [13], takes advantage of Unsorted Block Image (UBI) layer to provide good scalability and reliability [10]. While Flash-Friendly File System (F2FS) is designed to use the flash-based storage devices that already equip the hardware supported flash translation layer (to provide hardware-based WL) [11]. However, the latter two file systems (UBIFS and F2FS) are primarily designed for NAND flash. **Their key designs can hardly apply to NOR flash file system** because they either consume much of DRAM resource (UBIFS) or rely on hardware (F2FS).

Motivated by the above issues, we propose HNFFS, which takes advantage of unbounded memory design to gain high I/O performance, and applies several techniques to boost mount time (ESS), mitigate *Foreground GC* overheads (AGC), reduce memory usage (MT), and improve wear evenness (RSWL).

## III. Design and Implementation

### A. *File System Architecture*

The design goals of **Harmony NOR Flash File System (HNFFS)**, are to overcome the limitations of the state-of-the-art NOR flash file systems and be friendly to IoT devices. HNFFS is a log-structured file system that implemented based on JFFS2 architecture [1]. The basic architecture of HNFFS is shown in Fig. 1, and described as follows.

**Layout management**. HNFFS organizes the NOR flash layout as several erasable sectors, each sector contains a number of ***nodes***. The *node* describes one fragment of data in a file with its ***header*** and ***body***. The *header* contains the file number (the data belongs to), the logical offset (to that file), and the *node*'s length while the *body* carries the real file data. To fast locate the *node* in the NOR flash, during the mount operation, HNFFS scans the whole NOR flash, retrieves the *header*, and finally builds the ***inode table*** in-DRAM that maps the file number to a list of ***node infos*** that represent the in-flash *nodes*. Moreover, **HNFFS tallies each sector during the scanning to indicate the status of the sector and the log's tail**, and then it is ready to accept next I/O requests. We further propose ESS in Section III-B to boost the mount process by reserving *summary node* at the end of each sector, and scanning it to fast rebuild in-DRAM structures.

**File structure**. To fast index the file data, HNFFS builds a Red-Black (RB) tree [19] in-DRAM for each opened file. Each node in RB tree, called ***dnode***, contains a reference to the corresponding *node info*. The *dnode* is sorted by logical offset. The reason why we use RB tree is that it is well balanced and can perform efficient insertions/deletions, which is suitable for frequent file operations. However, the challenge is that RB tree consumes much of DRAM with the increment of the node number. which is unfriendly to IoT devices. By leveraging MT in Section III-D, HNFFS can significantly reduce the DRAM usage by dynamically merging sequential small-data writes.

**GC**. Due to the out-of-place updates, NOR flash will be quickly filled by a variety of *nodes*; thus, a GC mechanism must be supplied to solve the issue. Traditionally, GC first obtains one sector as the victim, then migrates all the valid *nodes* from the victim sector to other available sectors by copying and writing, and finally erases the victim sector.



Authorized licensed use limited to: Peking University. Downloaded on November 19,2022 at 09:09:10 UTC from IEEE Xplore. Restrictions apply.

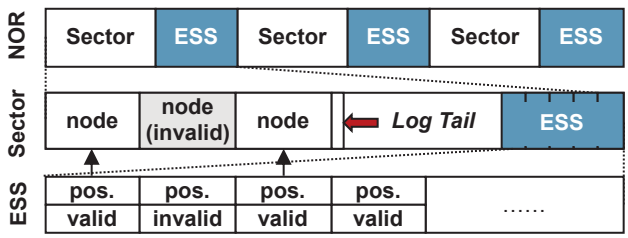


Fig. 2. HNFFS ESS architecture.

However, GC often brings heavy overheads due to frequent I/O and erase operations which blocks the write path significantly. To reduce the overheads, HNFFS proposes AGC by running parts of GC operation in advance, and thus offloads heavy GC burdens to the idle time, see details in Section III-C.

**Wear-Leveling**. Considering the limited erasing life-cycle of NOR flash, HNFFS carefully selects the victim sectors to provide an elegant Wear-Leveling. HNFFS selects the sector mainly by their dirty level. It categorizes sectors into three levels (based on the status of each sector aforementioned): (1) ***Free sector***. The sector contains nothing. (2) ***Clean sector***. The sector contains only valid *nodes*. (3) ***Dirty sector***. The sector contains invalid *nodes*. To handle the erase requests, HNFFS often selects the *dirty sector* as the victim. However, there is also a default probability of 5% (note that the probability is configurable) to select a *clean sector*. The selection of probability is based on the intuition that *clean sector* should be also erased for further usage to prevent uneven WL distribution, but since there are no invalid *node*s in it, GC can not benefit from it, thus we should have less chance to select *clean sector*. We then introduce the optimized WL algorithm of HNFFS (RSWL) in Section III-E, whose wear evenness is further enhanced by combining SWL strategy.

**Data atomicity**. HNFFS leverages the Cyclic Redundancy Checksum (CRC) to keep atomicity. HNFFS commits CRC code for each *node* and the incomplete *node* can be identified by verifying the CRC code. The reason why we not use smaller flags (e.g., 8 bits commit flag) to keep atomicity is that CRC can be also used to detect bit flip problem in NOR flash [20].

**Consistency**. A power failure may leave system in an inconsistency state. The log-structured file system is able to roll back to the most recent version to maintain its consistency: if a file is found to be inconsistent (e.g., the number of found data blocks are not equivalent to the data size recorded in its in-media inode), HNFFS first invalidates it, and then searches and validates the previous version of this inode; if the found inode is also inconsistent, HNFFS repeats the above operations until a consistent version is reverted. If there is no consistent version found, HNFFS marks the inode as broken, and each accessing to it results in an *EIO* error code.

### B. ESS Architecture

The mount time of HNFFS increases linearly as the NOR flash size increases, since HNFFS has to scan the whole NOR flash for rebuilding the in-DRAM *inode table* when mounting.

**ESS** is then proposed to mitigate the problem. The key design principal of ESS is that scanning as little as possible when mounting HNFFS. Fig. 2 shows the architecture of ESS. **The essence of ESS is that deploying *summary node* at the end of each NOR flash sector**. *Summary node* contains meta information (**meta entry** for brevity) of all the *node*s in the sector, e.g., valid flag, *node*'s position, etc. So, when HNFFS is mounting, it only takes a glance at small *summary node* of each sector, and then fast locates the valid *node*s' *header* according to the meta entries, and finally rebuilds the *inode table* to get ready to accept new I/O requests.

There are two operations for ESS: (1) **Allocation**. To allocate a new meta entry for one *node*, HNFFS first locates the corresponding slot in *summary node* by calculating the relative offset from *node* to the start of the sector where the *node* sits, and then validates the slot and fills it with the meta entry. (2) **Invalidation**. The process of invalidating one meta entry is similar to that of allocation. The difference is that we invalidates the located meta entry. Note that, in NOR flash, programming bit 1 (valid) to 0 (invalid) requires no erase operation, so the invalidation of meta entry do no harm to NOR flash's lifetime.

Incorporating ESS into HNFFS introduces the following two issues: one is space overheads, and the other is consistency.

- **Space overheads**. The space overheads of ESS are not significant since we only maintain the *node*'s reference (i.e., position) in it (rather than the whole corresponding *header*). In HNFFS, the size of *node* is aligned to 56B, and the corresponding meta entry in *summary node* is 8B. Assuming each sector contains at most $x$ *node*s, then HNFFS allocates $\frac{8xB}{8xB+56xB} \times 100\% = 12.5\%$ space for *summary node* in each sector. Note that, if we store *header* in ESS, although mount performance can be further improved due to less random read, the space overheads of ESS are increased to 36.4% (the size of *header* is 32B in HNFFS), which is high in the space-limited NOR flash.
- **Consistency**. We regard the *node*s as the true source of information to guarantee consistency between *summary node* and each *node*. If a crash occurs, we can always reconstruct *summary node*s by scanning corresponding sectors.

Our experimental results (see Section IV) show at most 1.6× performance improvement over JFFS2 when applying ESS mechanism to HNFFS.

### C. Adaptive Garbage Collection

We propose an **Adaptive GC (AGC) mechanism** to mitigate the overheads when GC is triggered. The key idea of AGC is combining *Foreground GC* and *Background GC*, putting the GC operation ahead of time to reduce the number of copies and writes of *Foreground GC* as much as possible.

***Foreground GC***. (1) **Trigger**. *Foreground GC* runs when a new write request is received and there are no enough space to reserve. Note that HNFFS pre-reserves five *free sector*s to enable GC's running. (2) **Victim selection**. The victim sectors (to be erased) are selected by RSWL algorithm (see Section III-E). The victim selection process will not stop until there are enough available space or no more invalid *nodes* in device. Note that the later situation causes an *ENOSPC* error code. (3) **Data Migration**. Once a victim sector is selected, *Foreground*



Authorized licensed use limited to: Peking University. Downloaded on November 19,2022 at 09:09:10 UTC from IEEE Xplore. Restrictions apply.

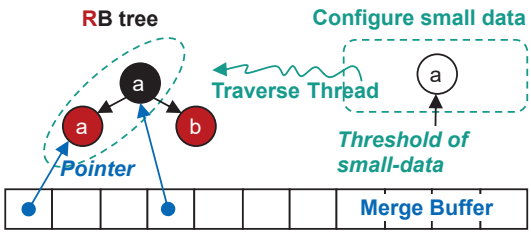


Fig. 3. Mergeable Tree (MT) structure

*GC* migrates all the valid *nodes* in it to the *free* or *clean sector*s by copying and writing, and finally erases the victim sector.

***Background GC***. (1) **Trigger**. *Background GC* is a thread that detects the system's status and triggers data migration when the system receives no write requests. However, running *Background GC* during idle time could be power consuming. To address the problem, HNFFS further restricts its trigger by setting the following two rules:

- *R1: Delay activation*. We set a *trigger threshold* (i.e., 30% of system utilization) to delay the activation of *Background GC* to the HNFFS's later runtime.
- *R2: Avoid aggressive running*. When the *Background GC* is activated, we manually set the *least time interval* between each trigger to 30ms to avoid running *Background GC* during system's idle time aggressively.

Note that the *trigger threshold* and the *least time interval* are both configurable. (2) **Data migration**. Considering the limited erasing life-cycle of NOR flash, *Background GC* performs no erase operations. Besides, as a background thread, *Background GC* should not consume too many CPU resources, migrating one *node* per pass is reasonable. Note that which sector is selected to perform migration is also determined from RSWL algorithm (see Section III-E).

Our experimental results show that AGC decreases 38–77% *Foreground GC* overheads compared to JFFS2 and brings higher write performance.

### D. Mergeable Tree

In the IoT devices, DRAM resource is restricted. In that case, a data structure should be designed carefully to efficiently utilize memory. Currently, HNFFS uses RB tree to index its files. Theoretically, to handle 10K times write, 10K *dnodes* are required for indexing. The challenge is that the size of write request is normally small in the real world [14]. In that case, **generating *dnode* for each (small) write is inefficient for memory usage and also degrades performance due to fast increasing tree height**.

To address the problem, we introduce the **MT mechanism**. The design principals of the MT mechanism are described as follows: (1) **Try not to index small-data**. Indexing small-data in HNFFS is inefficient, so MT tries to clean these small-data (by merging them) during runtime. (2) **Configure small-data dynamically**. Defining the threshold of small-data is not easy, because the size of data normally tends to be small ($\leq$ 4KB) in the real world [14]. So, MT regards the most frequent data as small-data. In this way, MT can potentially clean more *dnodes* in RB tree, which further reduces the memory usage and potentially increases the indexing performance.

TABLE II
NOR FLASH ENVIRONMENT FOR EXPERIMENTS.

| Experimental Environment[a] | NOR Flash Memory Characteristics |
|---|---|
| Platform : MINI2440 QEMU | Part Number : AM29LV160DB [15] |
| CPU : ARM9 101MHz | Sector Size : 16/8/8/32/64KB (35 Sectors) |
| Memory : SDRAM 64MB | Read Throughput : 8–14MB/s |
| OS : SylixOS 1.12.9 | Write Throughput : 9–279KB/s |

[a]Provided by SylixOS development kits.

The structure of MT is shown in Fig. 3. MT consists of two main components: RB tree and *merge buffer*. The *merge buffer* is a simple array that records pointers of the *dnodes* of small-data. The general workflow is described as follows:

- **Configuring and Identification**. MT configures small-data every hundred write requests by starting a *traverse thread* which scans the whole RB tree and takes the most frequent data size as the *threshold* of small-data (see "Configure small data" in Fig. 3). MT identifies the small-data by comparing the size of the request with the *threshold*.
- **Insertion**. If the request is exact small-data, MT inserts corresponding *dnode* into the RB tree and allocates a slot in the *merge buffer* to points to the *dnode* (see "Pointer" in Fig. 3). Note that if two *dnodes* occurs logical overlap (e.g., old *dnode* represents (0, 4K), new *dnode* represents (0, 8K)), then the old one will be freed.
- **Clean**. Once the *merge buffer* is full (no free slot to allocate), MT scans the buffer, merges the sequential writes (e.g., *dnodes* represent (0, 4K) and (4K, 8K)) into a large one (e.g., *dnode* represents (0, 8K)) and finally re-write the merged data into HNFFS. Since the new *dnode* evicts the old *dnodes* after insertion, the *dnodes* that represents small-data are cleaned. Note that HNFFS does not invalidate old small-data unless the new merged data is written and validated, so the consistency is still guaranteed.

Our experimental results show that the MT mechanism brings HNFFS 2.3× write performance improvement over JFFS2 and 6.04× over SPIFFS while reducing more than 90% memory usage of original RB tree.

### E. Random Static Wear-Leveling

WL is a critical concern when designing NOR flash file system since erasing life-cycle of NOR flash is limited and 10× less than that of NAND flash (See Table I). **RWL** and **SWL** are two main WL algorithms in state-of-the-art NOR flash file systems. However, they either provide poor wear evenness or lack the consideration of performance.

To address the problem, we introduce RSWL algorithm, which leverages the high performance of RWL and the reliability of SWL. The key design principles of RSWL are: (1) **Randomly** selecting sector level between *clean* and *dirty*. (2) **Carefully** comparing the erase count of the selected level sectors with the other level sectors to verify if the selection is OK. For example, RSWL selects *clean* level (5% probability, see **Wear-Leveling** in Section III), grabs one *clean sector* and compares its erase count with all the *dirty sector*s, and finally selects the lowest erase count sector among them. In that case, RSWL reduces the number of comparisons by randomly picking level and avoids unnecessary erasing of *clean sector*s



Authorized licensed use limited to: Peking University. Downloaded on November 19,2022 at 09:09:10 UTC from IEEE Xplore. Restrictions apply.

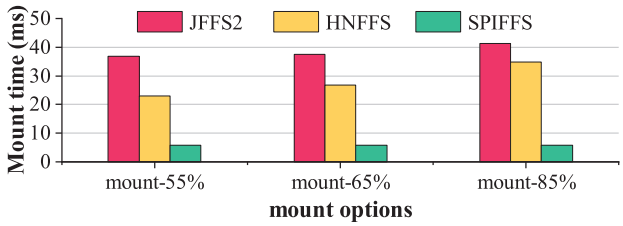


Fig. 4. Mount time comparison.

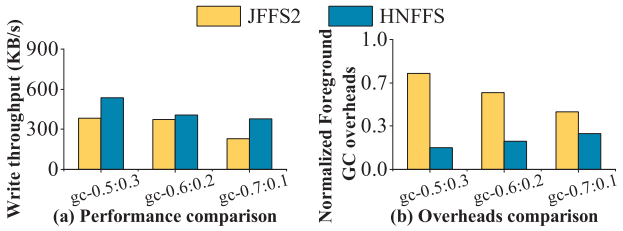


Fig. 5. GC comparison.

by comparing the static erase count with *dirty sector*s.

Our evaluation shows that RSWL gains 30% WL performance improvement over SWL while can improve wear evenness by as much as 60% compared to RWL.

## IV. Evaluation

In this section, we evaluate the performance of HNFFS. We compare HNFFS with JFFS2 and SPIFFS. We first implement HNFFS on SylixOS with kernel version 1.12.9, then we migrate JFFS2 and SPIFFS to SylixOS for further performance comparing. SylixOS is a powerful and open-source Real-Time Operating System (RTOS) that is designed to execute tasks in a guaranteed time [12]. The NOR flash memory system environment for experiments is based on QEMU [21], which is summarized in TABLE II. Note that HNFFS does not support multi threads currently, which is also costly in IoT scenarios.

At first, we compare the **mount time**. During the mount operation, flash file systems are required to collect the system-level information, so **mount time varies with the usage of file systems**. In that case, we design the experimental steps as follows: (1) **Mount** the file system. (2) **Create** a new file whose size is $S\%$ of the NOR flash (4KB per write). (3) **Remount** the file system.

Fig. 4 presents the mount time usage of difference experimental configurations (e.g., mount-55% indicates that creating a file whose size is 55% of NOR flash). In the figure, SPIFFS outperforms than any other file systems, that's mainly because SPIFFS doesn't need to build in-DRAM *inode table* when mounting [6]. We notice that HNFFS improves performance over JFFS2 by 1.2–1.6×, that's mainly contributed to ESS architecture, which reduces the scanning to NOR flash.

To understand what benefits AGC brings to HNFFS, we evaluate the write throughput when *Foreground GC* involved and the number of *nodes Foreground GC* migrates. **Note that we do not include SPIFFS in the GC experiment** because we mainly focus on how does HNFFS benefit from AGC mechanism compared to JFFS2. We design the experimental steps as follows: (1) **Create** a new file whose size is $S\%$ of the NOR flash. (2) **Delete** the file. (3) **Create** a new file again, whose size is $M\%$ of the NOR flash (4KB per write, one

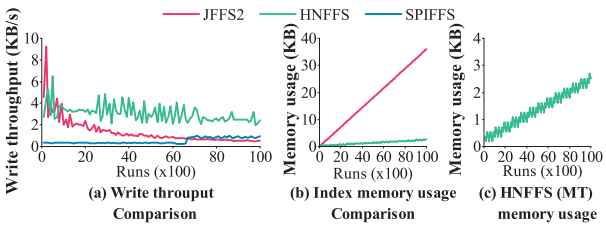


Fig. 6. Comparison of write performance and memory usage.

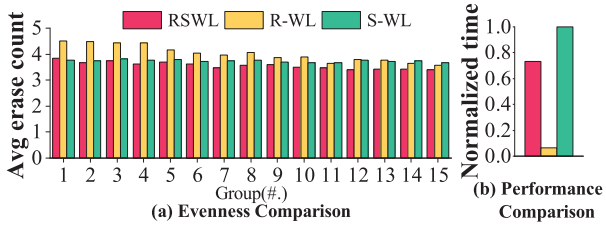


Fig. 7. Comparison of different WL algorithms. In figure (a), the variance of RSWL, SWL, and RWL is 0.12, 0.04, and 0.31, respectively.

second between each write request). This step makes sure the *Foreground GC* and *Background GC* will both be triggered.

We first compare the write throughput of step (3) in Fig. 5(a). The x-axis shows the different configurations of experiments (e.g., gc-0.5:0.3 indicates that creating a file whose size is 50% of NOR flash, deleting the file, and then creating 30% of which again). HNFFS shows the better write performance because *Background GC* has been migrated several *nodes* in advance and it consumes a few machine resources as well. We secondly compare the number of *nodes Foreground GC* migrates in JFFS2 and HNFFS. Fig. 5(b) shows the result. The result is normalized since we can regard it as a metric for *Foreground GC* overheads. In the figure, HNFFS reduces the migrated *nodes* by up to 38–77%, so does the overheads of *Foreground GC*, which gives a reasonable explanation for the better performance of HNFFS.

Next, we analyze the **memory usage and the write performance**. Note that, we measure the memory usage of indexing only for better observation. The experiment is conducted as follows: (1) **Create** a new empty file. (2) **Write** 16–128B data 10,000 times sequentially. For simplicity, the initial threshold of small-data is set to 128B, and each 100 writes contains at least 50 128B data (which means the threshold of small-data is always 128B).

The write throughput is shown in Fig. 6(a), while the comparison of memory usage between HNFFS (using MT) and JFFS2 (using RB tree) is shown in Fig. 6(b). Fig. 6(c) shows the detail of MT memory usage. In the Fig. 6(a), the write throughput of HNFFS outperforms JFFS2 by up to 2.3× and SPIFFS by up to 6.04×. At the same time, MT shows much lower memory usage in Fig. 6(b): the memory gap between RB tree and MT expands to 14.2× (i.e. MT saves 93% memory usage compared to RB tree). However, HNFFS suffers significant memory jitters as shown in Fig. 6(c) due to frequently merges and writes. Although HNFFS still leverages unbounded memory, the memory usage is reduced dramatically and is acceptably low in most cases.

Finally, we evaluate **the performance and wear even-**



Authorized licensed use limited to: Peking University. Downloaded on November 19,2022 at 09:09:10 UTC from IEEE Xplore. Restrictions apply.

**ness provided by different WL algorithms**. For better observation, we implement a 128MB NOR flash emulator on Windows (64KB per sector, 2048 sectors in total), and migrates *Foreground GC*, RWL (from JFFS2), SWL (from SPIFFS) and RSWL (from HNFFS) to it. The experiment is conducted as follows: (1) **Generate** workload that contains 384MB data. (2) **Randomly** write the data from workload until the NOR flash is fully filled with data. The invalidator is then triggered to randomly invalid *nodes* on NOR flash. Finally *Foreground GC* is called to leverage WL algorithms to clean sectors for the next write request.

Fig. 7(a) shows the comparison of the average erase count of per 136 sectors, the result suggests that RSWL performs less erase than RWL/SWL, that's mainly because RSWL has higher probability to choose *dirty sector*s as victims and carefully checks if the sector should be erased by comparing erase count. Note that the variance of RSWL is 0.12 which is higher than SWL (0.04) but far less than RWL (0.31), which indicates that RSWL improves wear evenness by as much as 60% compared to RWL. Fig. 7(b) shows the comparison of WL performance (the results are normalized), which suggests that RSWL performs 30% more efficiently than SWL. **Performance gap between RSWL and RWL is acceptable since wear evenness is more important in NOR flash file system**.

## V. Discussions

**Energy efficiency**. Energy efficiency is a popular topic in IoT scenarios [28]. However, the *Background GC* mechanism of HNFFS might cause unnecessary battery drainage during system's long periods of idle time. Although we have set two rules to restrict the trigger of *Background GC* to mitigate the problem (see *R1: Delay activation* and *R2: Avoid aggressive running* in Section III-C), the unnecessary power consumption still remains. We think this can be further optimized by applying an idle time detector to disactive *Background GC* when the system is idle for long time, as our future work.

**Tradeoff in mount operation**. In the Fig. 4, the mount process of SPIFFS is much faster than HNFFS since it does not build in-DRAM index. HNFFS can also achieve comparable performance as SPIFFS by lazily constructing its in-DRAM structures at runtime. However, such lazy strategy causes drastically I/O performance degradation due to the large overheads of scanning for rebuilding at runtime. So, HNFFS sacrifices some mount performance to prevent significant latency in the I/O path, which is reasonable.

## VI. Conclusion

In this paper, we revisit the design of two representative NOR flash file systems and propose HNFFS, a NOR flash file system that aims to mitigate the problems of existing NOR flash file systems. HNFFS is aware of both performance and the erasing life-cycle of NOR flash. It applies ESS, AGC, and RSWL techniques, and explores the way to reduce memory usage by proposing MT mechanism, which is friendly to IoT devices. Our evaluation results underscore how our design principles and tradeoffs lead to performance advantages, over other existing NOR flash file systems.


## Acknowledgement

This work was supported in part by NSFC under Grants 61972441 and 62141213, Shenzhen Science and Technology Program under Grant JCYJ20200109113427092, Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies (2022B1212010005), and Shanghai Science and Technology Project 22QA1403300. Wen Xia (xiawen@hit.edu.cn) is the corresponding author.


## References


[1] Woodhouse, D. "JFFS: The Journalling Flash File System". Red Hat, Inc. White Paper, 2001.

[2] F. Douglis et al., "Storage Alternatives for Mobile Computers," In Proc. OSDI, 1994.

[3] Lai, S., "Brief History of ETOX™ NOR Flash Memory". Journal of Nanoscience and Nanotechnology, 12(10), pp.7597-7603, 2012.

[4] H. -T. Lue et al., "A Vertical 2T NOR (V2T) Architecture to Enable Scaling and Low-Power Solutions for NOR Flash Technology," 2020 IEEE Symposium on VLSI Technology, 2020, pp. 1–2.

[5] Seung-Ho Lim et al., "An efficient NAND flash file system for flash memory storage," in IEEE Transactions on Computers, vol. 55, no. 7, pp. 906-912, July 2006.

[6] Peter Andersson et al., SPIFFS, https://github.com/pellepl/spiffs, 2013.

[7] JoEmbedded et al., JesFS, https://github.com/joembedded/JesFs, 2018.

[8] L. Chang, P. Sung and P. Chen, "Fast file synching for applications in flash-based Android devices, in Proc. IEEE 3th NVMSA, 2014.

[9] Huawei LiteOS. 2021. Accessed: September. 07, 2021. [Online]. Available at: https://www.huawei.com/minisite/liteos/en/index.html.

[10] Schierl A et al., (2009) Abstract Specification of the UBIFS File System for Flash Memory.

[11] C. Lee et al., F2FS: A New File System for Flash Storage. In Proc. FAST '15, 2015.

[12] Y. Zhang et al., "Brief Industry Paper: SylixOS: A Secure and Compatible RTOS with Constant Scheduling on SMP," in Proc. RTAS, 2021.

[13] BITYUCKIY, A. B. 2005. JFFS3 design issues. http://www.linux.mtd.infradead.org/doc/JFFS3design.pdf.

[14] Ji, C et al., Pattern-Guided File Compression with User-Experience Enhancement for Log-Structured File System on Mobile Devices. In Proc. FAST '21, 2021.

[15] AMD, Am29LV160D Datasheet, 2000.

[16] LittleFS. 2022. Accessed: April. 21, 2022. [Online]. Available at: https://github.com/littlefs-project/littlefs.

[17] Q-FLASH MEMORY. 2022. Accessed: April. 21, 2022. [Online]. Available at: https://pdf1.alldatasheet.com/datasheet-pdf/view/75850/MICRON/MT28F128J3.html.

[18] NAND Flash Memory. 2022. Accessed: April. 21, 2022. [Online]. Available at: https://pdf1.alldatasheet.com/datasheet-pdf/view/513742/MICRON/MT29F2G01ABAGDSF.html.

[19] Cormen, Thomas H et al., 2001. "Red-Black Trees". Introduction to Algorithms (second ed.). MIT Press. pp. 273-301.

[20] A. Menu et al., "Single-bit Laser Fault Model in NOR Flash Memories: Analysis and Exploitation," 2020 Workshop on Fault Detection and Tolerance in Cryptography (FDTC), 2020.

[21] Bellard et al., "QEMU, a fast and portable dynamic translator." USENIX annual technical conference, FREENIX Track. Vol. 41. No. 46. 2005.

[22] Aviad Zuck et al., NANDFS: a flexible flash file system for RAM-constrained systems. In Proc. EMSOFT '09. 2019.

[23] Youyou Lu et al., ReconFS: A Reconstructable File System on Flash Storage. In Proc, FAST '14, 2014.

[24] Liu et al., Optimizing NAND flash-based SSDs via retention relaxation. In Proc. FAST '12, 2012.

[25] L. Han et al., "Rebirth-FTL: Lifetime optimization via Approximate Storage for NAND Flash," in Proc. IEEE 8th NVMSA, 2019.

[26] Christian Zimmermann. Mobile Phone Forensics: Analysis of the Android Filesystem (YAFFS2). Master's thesis, University of Mannheim, 2011.

[27] Yaffs Direct Interface. 2022. Accessed: April. 25, 2022. [Online]. Available at: https://yaffs.net/documents/yaffs-direct-interface.

[28] N. Kaur et al., "An Energy-Efficient Architecture for the Internet of Things (IoT)," in IEEE Systems Journal, vol. 11, no. 2, pp. 796-805, June 2017.




Authorized licensed use limited to: Peking University. Downloaded on November 19,2022 at 09:09:10 UTC from IEEE Xplore. Restrictions apply.